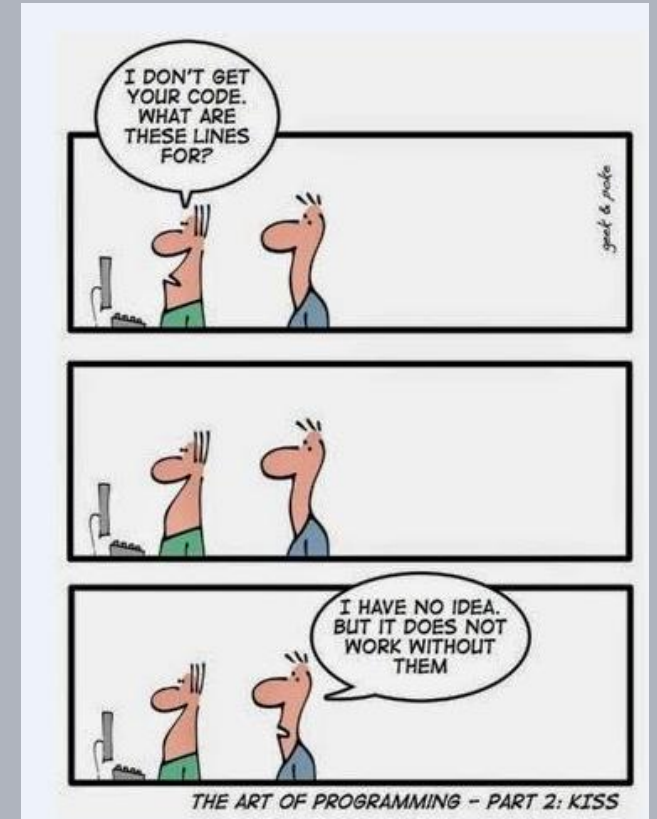


Advanced Software Design Techniques

Fundamentals of Software Quality and Clean Architecture

Prof. Dr.-Ing. Peter Fromm



Content

- What makes an Architecture a Good Architecture
- Behaviour versus Structure
- IEC9126 Quality Modell
 - Functionality
 - Reliability
 - Usability
 - Maintainability
 - Portability
 - Efficiency
- Motivation and Need for Refactoring
- Incremental, Iterative development

How do I write Clean Code?



Introduction

“It doesn’t take a huge amount of knowledge and skill to get a program working. Kids in school do it all the time...The code they produce may not be that pretty; but it works. It works because getting something to work – once – just isn’t that hard.

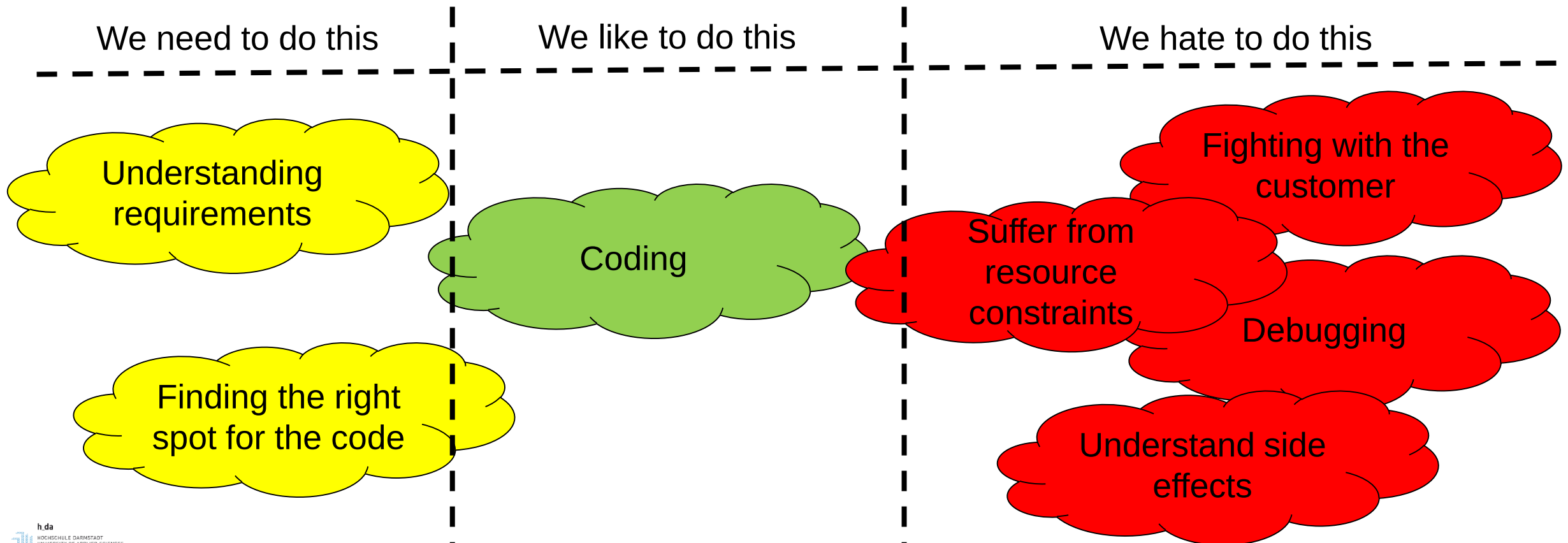
Getting it right is another matter entirely. Getting software right is hard. It takes knowledge and skills that most young developers haven’t yet acquired. It requires thought and insight that most programmers don’t take the time to develop.”

Robert C. Martin / Clean Architecture

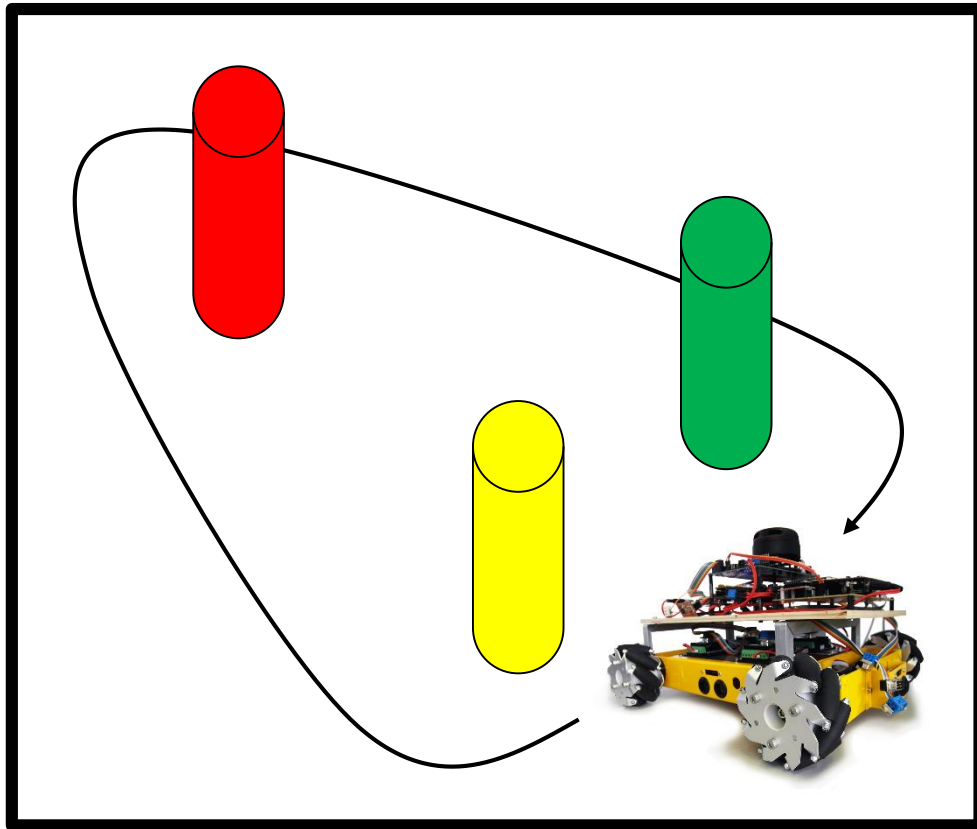
Motivation

“The goal of software architecture is to minimize the human resources required to build and maintain the required system.”

Robert C. Martin / Clean Architecture



Teamproject



Good job?

Worth 300.000€?

Worth 75.000€?

<https://www.youtube.com/watch?v=XJ5ApJoVQBA>

Backlog

- Planned duration 3 month
 - Month 1 – Concept
 - Month 2 – Implementation
 - Month 3 – Qualification and Documentation
- First concept after 2 month
- First (partly working) prototype after 3 month
- Exam break
- Waiting for a miracle in month 5

Understanding
existing parts

Agreement of
requirements

No real prototyping,
no consolidation
afterwards

Focus on irrelevant
(but easy) parts

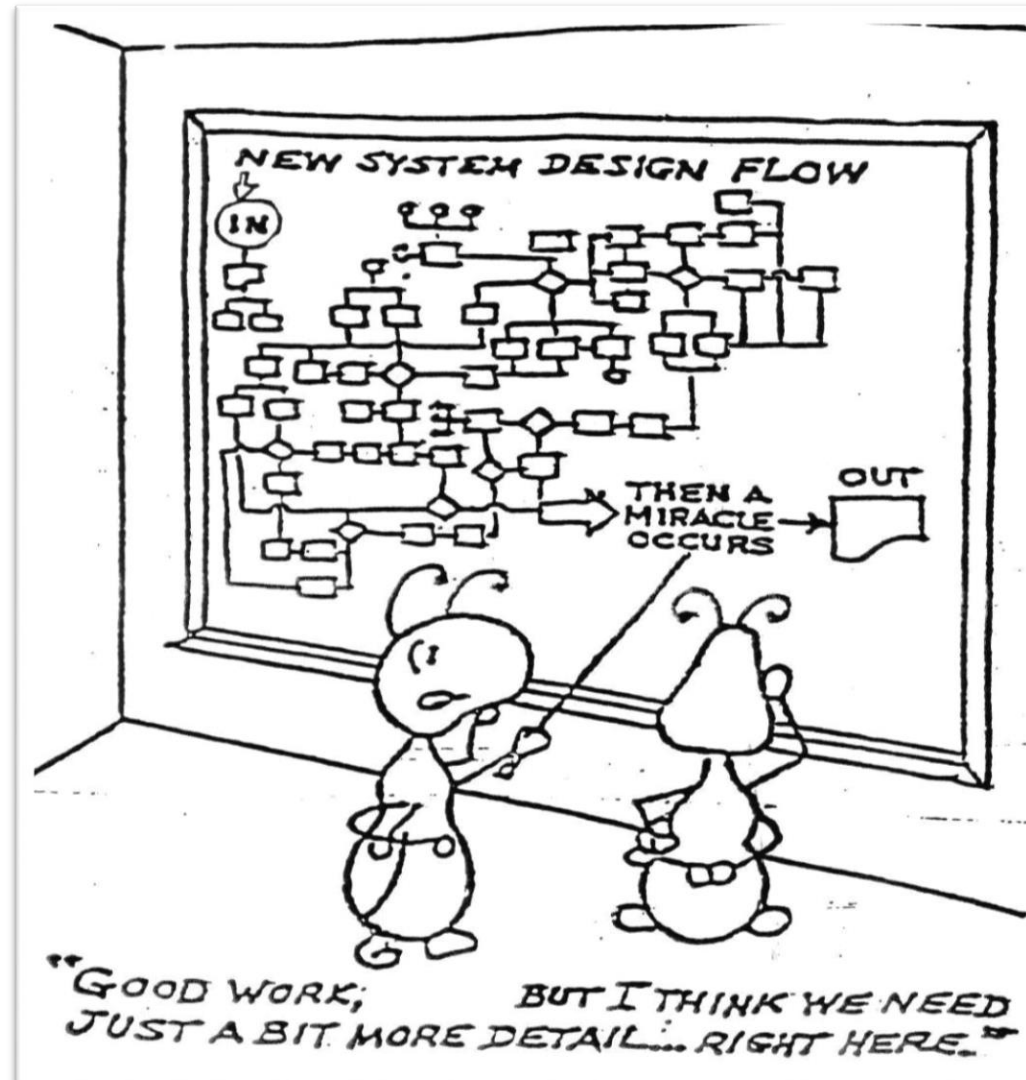
Limited architectural
focus

Underestimated
hardware complexity

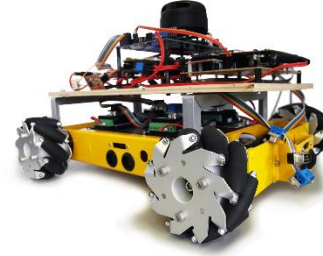
No risk mitigation
(e.g. sensor
reliability)

Undefined team
responsibilities

Real life...



A system engineering perspective



Values:

Behaviour (WHAT)

Structure (HOW)

Marketing

Change

Repair

Extend

Soft-ware: ware = system. Soft = we can EASILY build and change it.

A somewhat drastic perspective

Good behaviour but bad structure

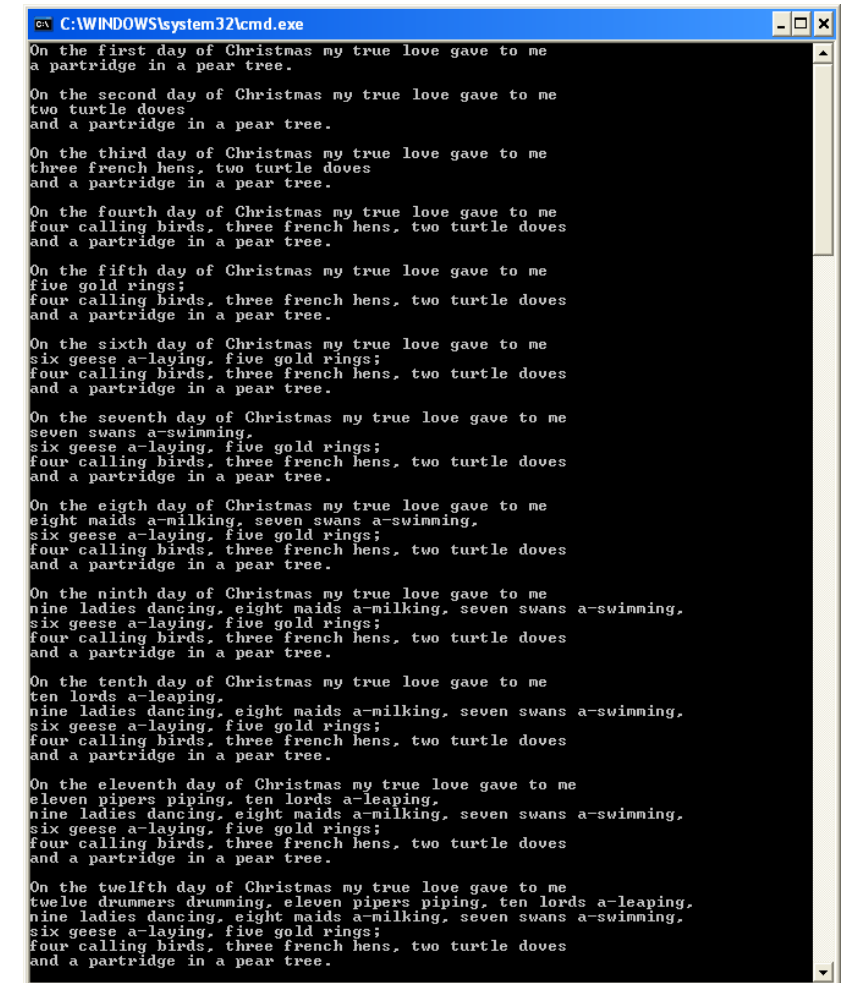
- The customer will be happy, until the first change.
- Then, the solution will be worthless.

Wrong behaviour but good structure

- The customer is unhappy and complains
- We can easily fix it
- The customer is happy

17 LOC.

Let's change Christmas to
Weihnachten...



```
C:\WINDOWS\system32\cmd.exe
On the first day of Christmas my true love gave to me
a partridge in a pear tree.

On the second day of Christmas my true love gave to me
two turtle doves
and a partridge in a pear tree.

On the third day of Christmas my true love gave to me
three french hens, two turtle doves
and a partridge in a pear tree.

On the fourth day of Christmas my true love gave to me
four calling birds, three french hens, two turtle doves
and a partridge in a pear tree.

On the fifth day of Christmas my true love gave to me
five gold rings;
four calling birds, three french hens, two turtle doves
and a partridge in a pear tree.

On the sixth day of Christmas my true love gave to me
six geese a-laying, five gold rings;
four calling birds, three french hens, two turtle doves
and a partridge in a pear tree.

On the seventh day of Christmas my true love gave to me
seven swans a-swimming,
six geese a-laying, five gold rings;
four calling birds, three french hens, two turtle doves
and a partridge in a pear tree.

On the eighth day of Christmas my true love gave to me
eight maids a-milking, seven swans a-swimming,
six geese a-laying, five gold rings;
four calling birds, three french hens, two turtle doves
and a partridge in a pear tree.

On the ninth day of Christmas my true love gave to me
nine ladies dancing, eight maids a-milking, seven swans a-swimming,
six geese a-laying, five gold rings;
four calling birds, three french hens, two turtle doves
and a partridge in a pear tree.

On the tenth day of Christmas my true love gave to me
ten lords a-leaping,
nine ladies dancing, eight maids a-milking, seven swans a-swimming,
six geese a-laying, five gold rings;
four calling birds, three french hens, two turtle doves
and a partridge in a pear tree.

On the eleventh day of Christmas my true love gave to me
eleven pipers piping, ten lords a-leaping,
nine ladies dancing, eight maids a-milking, seven swans a-swimming,
six geese a-laying, five gold rings;
four calling birds, three french hens, two turtle doves
and a partridge in a pear tree.

On the twelfth day of Christmas my true love gave to me
twelve drummers drumming, eleven pipers piping, ten lords a-leaping,
nine ladies dancing, eight maids a-milking, seven swans a-swimming,
six geese a-laying, five gold rings;
four calling birds, three french hens, two turtle doves
and a partridge in a pear tree.
```

Old school design quality

A well-defined segmentation of the project effort ensures system modularity . Each task forms a separate, distinct program module. At implementation time each module and its inputs and outputs are well defined , there is no confusion in the intended interface with other system modules. At checkout time the integrity of the module is tested independantly ; there are few scheduling problems in synchronizing the completion of several tasks before checkout can begin. Finally, the system is maintained in modular fashion; system errors and deficiencies can be traced to specific system modules, thus limiting the scope of detailed error searching.

Gouthier and Pont, 1970

Bad design...

Rigidity

- Rigidity is the tendency for software [to be difficult to change](#), even in simple ways. Every change causes a cascade of subsequent changes in dependent modules. What begins as a simple two day change to one module grows into a multi-week marathon of change in module after module as the engineers chase the thread of the change through the application.

Fragility

- Closely related to rigidity is fragility. Fragility is the tendency of the software [to break in many places every time it is changed](#). Often the breakage occurs in areas that have no conceptual relationship with the area that was changed. Such errors fill the hearts of managers with foreboding. Every time they authorize a fix, they fear that the software will break in some unexpected way.

Immobility

- Immobility is the [inability to reuse software](#) from other projects or from parts of the same project. It often happens that one engineer will discover that he needs a module that is similar to one that another engineer wrote. However, it also often happens that the module in question has too much baggage that it depends upon. After much work, the engineers discover that the work and risk required to separate the desirable parts of the software from the undesirable parts are too great to tolerate. And so the software is simply rewritten instead of reused.

Viscosity

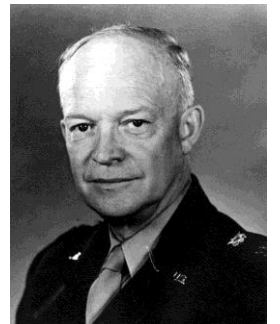
- Viscosity comes in two forms: [viscosity of the design, and viscosity of the environment](#). When faced with a change, engineers usually find more than one way to make the change. Some of the ways preserve the design, others do not (i.e. they are hacks). When the design preserving methods are harder to employ than the hacks, then the viscosity of the design is high. It is easy to do the wrong thing, but hard to do the right thing.

Eisenhower Matrix

„I have two kind of problems, the urgent and the important. The urgent are not important, and the important are never urgent“

Eisenhower, from a speech at Northwestern University in 1954

- Feature, behaviour is typically urgent.
- Structure is important

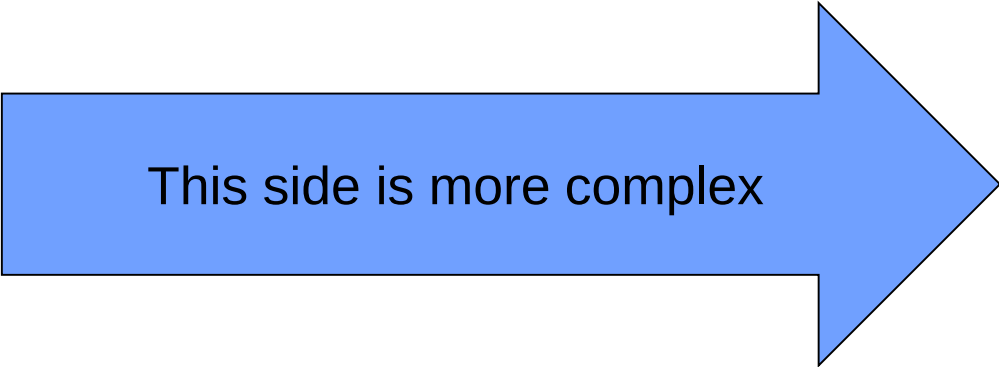


Priority	Matrix	Example
1	Urgent and important	E.g. safety and security features
2	Not urgent and important	Software Architecture
3	Urgent but not important	Relevant customer features
4	Not urgent and not important	Nice to have features

Behaviour versus structure

Behaviour

- Understand functional requirements
- Translate requirements into code



This side is more complex

Structure

- Understand functional and non-functional requirements
- Identify components and interfaces
 - Hardware
 - Software components
 - Classes
 - Functions
- Decide about central services (e.g. error handling, security, safety)
- Make or buy
- Decision of architectural and coding patterns

Stairway to heaven (hopefully)

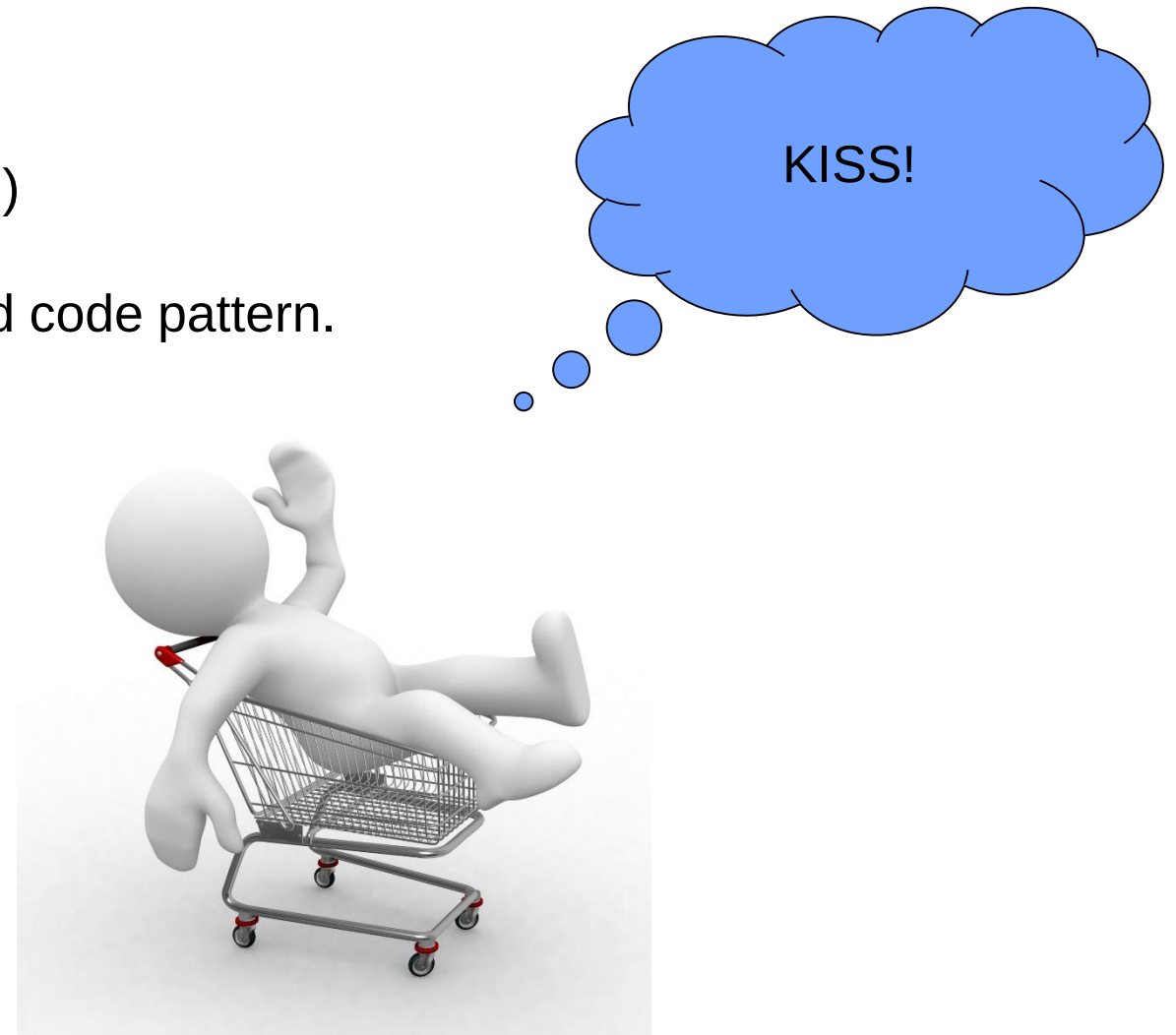
- 1. Distribute functionality on components (hardware, software components, classes/files, functions)
2. Define interfaces and dependencies
3. Identify configurable / generic parts
4. Proof of concept - skinny sheep
- 5. Implement behaviour
 - a) Add feature (using coding pattern and guidelines)
 - b) Test feature
 - c) Integrate feature
 - d) Evaluate code and architecture
- 6. Refactor if needed



Zooming in the structure question

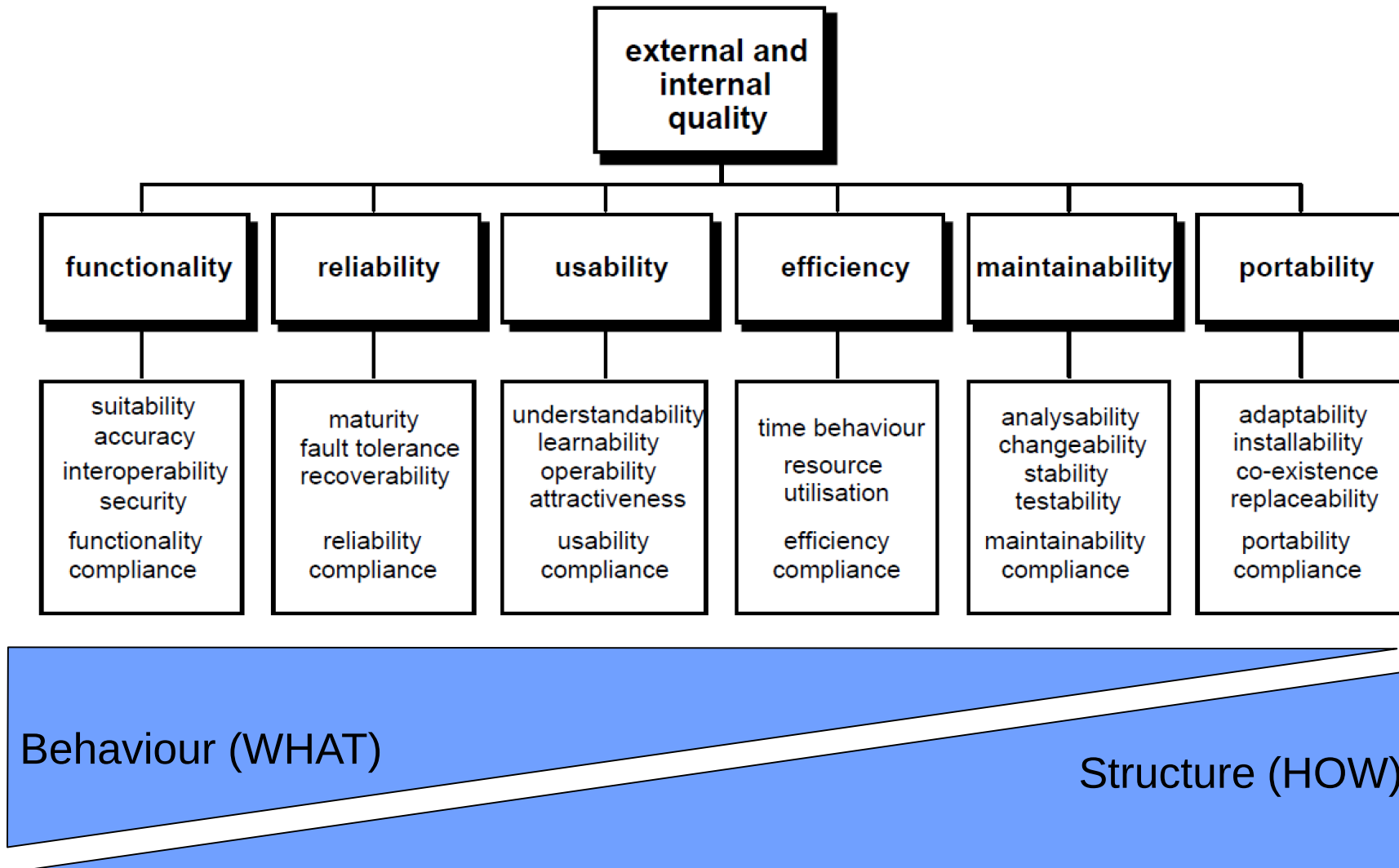
What makes a design a good design?

- Good Documentation (Comments)
- Good structure (strong cohesion, low coupling)
- Unique and easy to use interfaces
- Understandable and risk mitigating design and code pattern.



A more formal definition of software quality

IEC9126 – the –ilities...



External and Internal Quality - Examples

External quality: Specifies the required level of quality from the external (user) view

- Functionality - are all required features available
- Reliability - does the system work as intended, for a sufficient long lifetime
- Useability - is the product easy to use
- Efficiency - does the user get responses as fast as possible
- Maintainability - are updates easily possible
- Portability - does it work on my notebook and tablet?

External and Internal Quality - Examples

Internal quality: Specifies the required level of quality from the internal (developer) view

- Functionality
 - good cohesion of components, classes and functions
- Reliability
 - good error handling, avoidance of overruns, buffer overflows
- Useability
 - good interface design
- Efficiency
 - runtime of functions, latencies of signals, memory consumption
- Maintainability
 - do I find the place for a change, can components easily be exchanged
- Portability
 - avoidance of platform specific code (e.g. data type sizes)

Another nice diagram from the IEC9126...

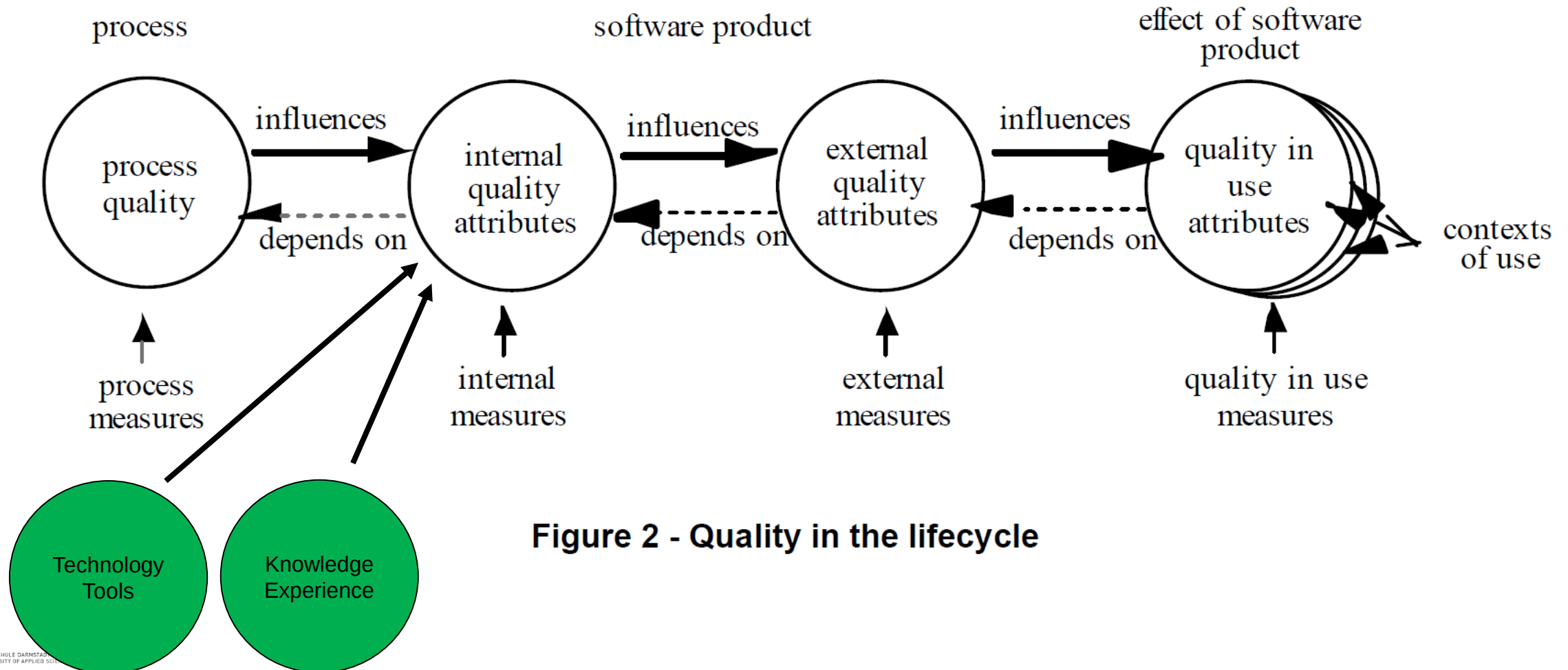


Figure 2 - Quality in the lifecycle

Our example story

Some years ago – the First Lidar Prototype



And the corresponding code...

```
119 static uint8 Lidar_ResponseLength = 0;
120 static uint8 Lidar_RxDescriptorFlag;
121 static int8 Lidar_CurrentBufferIndex = -1;
122 static ResponseBuffer_t Lidar_RxBuffer[NO_OF_SCAN];
123 static ResponseBuffer_t Lidar_RxDescriptor;
124 static uint8 NewScanStartedFlag = 0, NewScanFlag = 0;
125 static uint8 NewScan_quality = 0;
126 static uint8 NewScan_Angle = 0;
127 static uint64 NumberOfReceivedBytes = 0;
128 volatile static ReqType_t Lidar_RequestType;
129 static volatile uint32 Lidar_Timer1msCount = 0;
130 static volatile boolean_t Lidar_ErrorFlag = FALSE;
131 volatile uint8 response_recieved = 0x00;
132 static uint8 Lidar_RetryCount = 0;
133 static uint8 Lidar_statupFlag = 0;
```

No FSM
pattern used

Not
threadsafe

Hard to
maintain

Magic
numbers

Low performance
due to busy waiting

The code is working.
But is it good?

We have
comments

Names are
not too bad

```
257 void Lidar_UpdateRxBuffer(uint8 data)
258 {
259     if (Lidar_statupFlag == 1)
260     {
261         // check if the response descriptor is received
262         if (Lidar_RxDescriptorFlag == 1)
263         {
264             // check if it a scan request
265             if (Lidar_ResponseLength == 255)
266             {
267                 //DEBUG_PRINT("Scan");
268                 // check for new scan
269                 Lidar_CheckForNewScan(data);
270                 //check if new scan is started
271                 if ((NewScanFlag == 0) && (NewScanStartedFlag == 1))
272                 {
273                     // update the scan value in buffer
274                     Lidar_RxBuffer[Lidar_CurrentBufferIndex].Buffer[Lidar_RxBuffer[Lidar_CurrentBufferIndex].Highindex++] = data;
275                     Lidar_RxBuffer[Lidar_CurrentBufferIndex].BufferFillLevel++;
276                 }
277                 // increment the nuber of bytes receivedd
278                 NumberOfReceivedBytes++;
279             }
280             // all other request other than scan
281             else
282             {
283                 Lidar_CurrentBufferIndex = 0;
284                 Lidar_RxBuffer[Lidar_CurrentBufferIndex].Buffer[Lidar_RxBuffer[Lidar_CurrentBufferIndex].Highindex++] = data;
285                 Lidar_RxBuffer[Lidar_CurrentBufferIndex].BufferFillLevel++;
286                 if (Lidar_RxBuffer[Lidar_CurrentBufferIndex].BufferFillLevel == Lidar_ResponseLength)
287                 {
288                     // DEBUG_PRINT("Health");
289                     Lidar_RxBuffer[Lidar_CurrentBufferIndex].BufferStatusFlag = Completed;
290                     response_recieved = 0x01;
291                 }
292             }
293         }
294         // receive response descriptor
295         if ((Lidar_RxDescriptorFlag == 0) && (Lidar_ResponseLength > 0x00))
296         {
297             NumberOfReceivedBytes++;
298             if (NumberOfReceivedBytes == 0x07)
299             {
300                 Lidar_RxDescriptorFlag = 1;
301                 NumberOfReceivedBytes = 0;
```

Internal quality affects external quality

- Although the sensor rotated at 5Hz, the data was only updated with 2Hz
- The system ran for typically a few minutes, then the application stopped – the synchronisation of the UART data stream coming from the sensor got lost.

Refactoring attempt 1

- Separate protocol handling and data processing in 2 separate tasklets to reduce busy waiting
- Did not work due to missing thread safety

Refactoring attempt 2

- Create improved data structure with double buffer
- Introduce RTOS
- But keep parts of the code base (due to time constraints)
- It worked, somehow....

Refactoring attempt 3

- Complete redesign and introduction of C++



Functionality



Functionality

The goal is to achieve the necessary and sufficient quality to meet the real needs of users. ISO 8402 defines quality in terms of the ability to satisfy stated and implied needs. However, needs stated by a user do not always reflect the real user needs, because:

- (1) a user is often not aware of his real needs,
- (2) needs may change after they are stated,
- (3) different users may have different operating environments, and
- (4) it may be impossible to consult all the possible types of user, particularly for off the-shelf software.

So quality requirements cannot be completely defined before the beginning of design. Yet, it is necessary to understand the real user needs in as much detail as possible, and represent these in the requirements. The goal is not necessarily to achieve perfect quality, but the necessary and sufficient quality for each specified context of use when the product is delivered and actually used by users.

IEC9126

Functionality

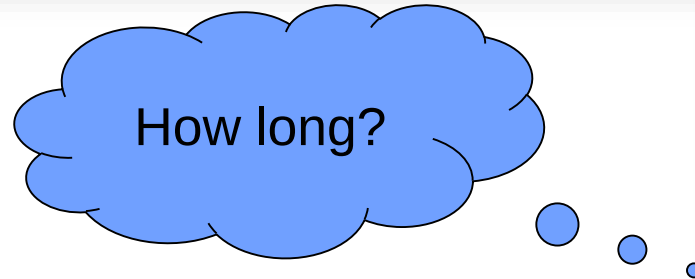
6.1 Functionality

The capability of the software product to provide functions which meet stated and implied needs when the software is used under specified conditions.

NOTE 1 This characteristic is concerned with what the software does to fulfil needs, whereas the other characteristics are mainly concerned with when and how it fulfils needs.

NOTE 2 For the stated and implied needs in this characteristic, the note to the definition of quality in B.21 applies.

NOTE 3 For a system which is operated by a user, the combination of functionality, reliability, usability and efficiency can be measured externally by quality in use (see clause 7).



6.1.1 Suitability

The capability of the software product to provide an appropriate set of functions for specified tasks and user objectives.

NOTE 1 Examples of appropriateness are task-oriented composition of functions from constituent sub-functions, and capacities of tables.

NOTE 2 Suitability corresponds to suitability for the task in ISO 9241-10.

NOTE 3 Suitability also affects operability.

6.1.2 Accuracy

The capability of the software product to provide the right or agreed results or effects with the needed degree of precision.

6.1.3 Interoperability

The capability of the software product to interact with one or more specified systems.

NOTE Interoperability is used in place of compatibility in order to avoid possible ambiguity with replaceability (see 6.6.4).

6.1.4 Security

The capability of the software product to protect information and data so that unauthorised persons or systems cannot read or modify them and authorised persons or systems are not denied access to them.

NOTE 2 **Safety** is defined as a characteristic of quality in use, as it does not relate to software alone, but to a whole system.

6.1.5 Functionality compliance

The capability of the software product to adhere to standards, conventions or regulations in laws and similar prescriptions relating to functionality.

Lidar example – Functionality Pitfalls

Awareness

- The protocol of the Lidar must be received and translated into a scan.
- Check for a possible collision in the driving direction of the car.
- The collision information shall be transmitted through another UART connection.

Missing awareness

- New versions of the Lidar sensor provided new protocol definitions.
- In addition to the anti collision algorithm, additional services like fullscan transmission for navigation purposes where required.
- What is a good resolution for the scan data?
- How can invalid sensor values be detected?

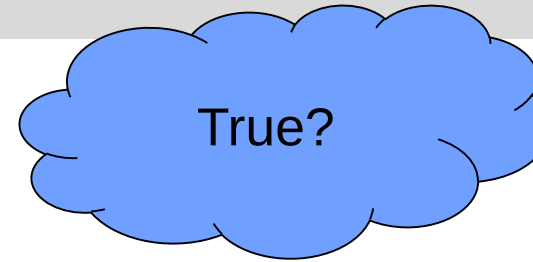
Reliability

6.2 Reliability

The capability of the software product to maintain a specified level of performance when used under specified conditions.

NOTE 1 Wear or ageing does not occur in software. Limitations in reliability are due to faults in requirements, design, and implementation. Failures due to these faults depend on the way the software product is used and the program options selected rather than on elapsed time.

NOTE 2 The definition of reliability in ISO/IEC 2382-14:1997 is "The ability of functional unit to perform a required function...". In this document, functionality is only one of the characteristics of software quality. Therefore, the definition of reliability has been broadened to "maintain a specified level of performance..." instead of "...perform a required function".



6.2.1 Maturity

The capability of the software product to avoid failure as a result of faults in the software.

A large blue thought bubble with a black outline, containing the text "Making software robust and reliable is a huge challenge → extra lecture!". It is connected to the main text area by a small blue dot.

Making software robust and
reliable is a huge challenge
→ extra lecture!

6.2.2 Fault tolerance

The capability of the software product to maintain a specified level of performance in cases of software faults or of infringement of its specified interface.

NOTE The specified level of performance may include fail safe capability.

6.2.3 Recoverability

The capability of the software product to re-establish a specified level of performance and recover the data directly affected in the case of a failure.

NOTE 1 Following a failure, a software product will sometimes be down for a certain period of time, the length of which is assessed by its recoverability.

NOTE 2 **Availability** is the capability of the software product to be in a state to perform a required function at a given point in time, under stated conditions of use. Externally, availability can be assessed by the proportion of total time during which the software product is in an up state. Availability is therefore a combination of maturity (which governs the frequency of failure), fault tolerance and recoverability (which governs the length of down time following each failure). For this reason it has not been included as a separate subcharacteristic.

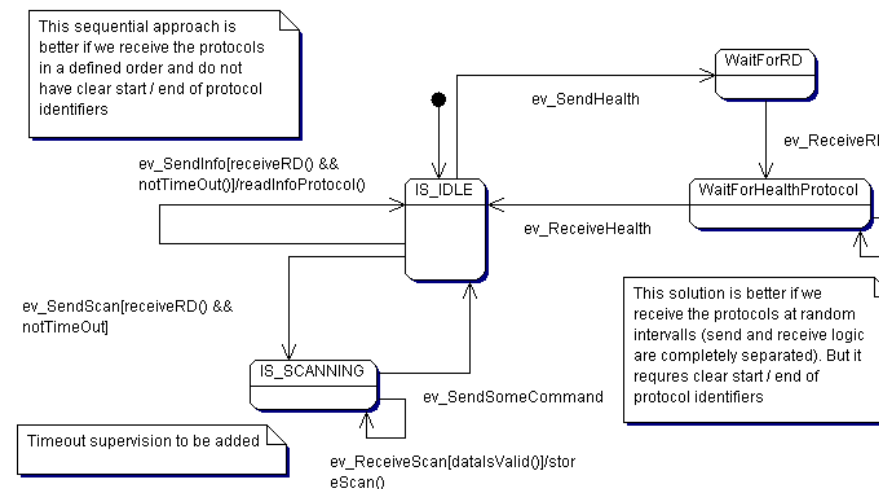
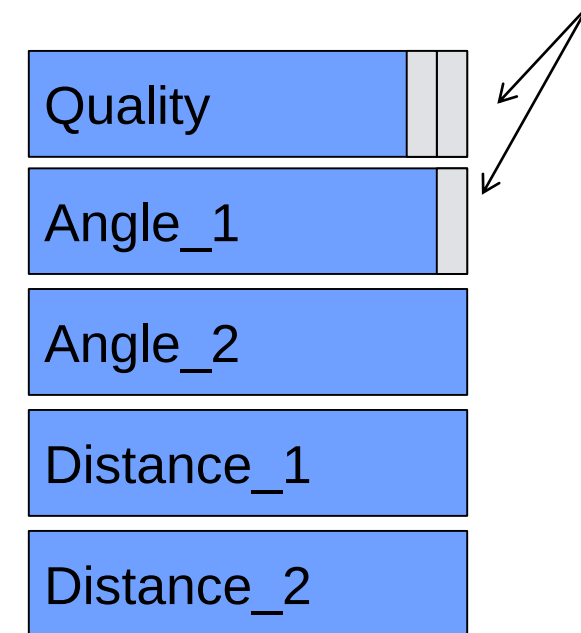
6.2.4 Reliability compliance

The capability of the software product to adhere to standards, conventions or regulations relating to reliability.

Lidar Example – Reliability Pitfalls

- Missing awareness of synchronisation challenge – 3 bits in a data stream of several 100-1000 Bytes.
- In case of a lost synchronisation, how can we resync?
- How can we reduce the risk of a lost synchronisation?

Synchronization Bits



Usability

6.3 Usability

The capability of the software product to be understood, learned, used and attractive to the user, when used under specified conditions.

NOTE 1 Some aspects of functionality, reliability and efficiency will also affect usability, but for the purposes of ISO/IEC 9126 they are not classified as usability.

NOTE 2 Users may include operators, end users and indirect users who are under the influence of or dependent on the use of the software. Usability should address all of the different user environments that the software may affect, which may include preparation for usage and evaluation of results.

6.3.4 Attractiveness

The capability of the software product to be attractive to the user.

NOTE This refers to attributes of the software intended to make the software more attractive to the user, such as the use of colour and the nature of the graphical design.

6.3.5 Usability compliance

The capability of the software product to adhere to standards, conventions, style guides or regulations relating to usability.

6.3.1 Understandability

The capability of the software product to enable the user to understand whether the software is suitable, and how it can be used for particular tasks and conditions of use.

NOTE This will depend on the documentation and initial impressions given by the software.

6.3.2 Learnability

The capability of the software product to enable the user to learn its application.

NOTE The internal attributes correspond to suitability for learning as defined in ISO 9241-10.

6.3.3 Operability

The capability of the software product to enable the user to operate and control it.

NOTE 1 Aspects of suitability, changeability, adaptability and installability may affect operability.

NOTE 2 Operability corresponds to controllability, error tolerance and conformity with user expectations as defined in ISO 9241-10.

NOTE 3 For a system which is operated by a user, the combination of functionality, reliability, usability and efficiency can be measured externally by quality in use.

Lidar Example – Usability Pitfalls

- Missing diagnostics in case of problems
- No recovery option
- Unclear signal flow / interfaces to external ECU's:
 - What is the driving direction of the car and how can it be determined?
 - What happens if the car is stopped?
 - What is the proper ILO cycle?



Side notice

Functionality versus Usability



Recent newspaper
article: most
accidents are caused
by driver distractions.



80/20 rule

In 80% of all cases you will use 20% of the features

80% of the features require 20% of the effort, adding the remaining 20% will create 80% of the effort.

6.4 Efficiency

The capability of the software product to provide appropriate performance, relative to the amount of resources used, under stated conditions.

NOTE 1 Resources may include other software products, the software and hardware configuration of the system, and materials (e.g. print paper, diskettes).

NOTE 2 For a system which is operated by a user, the combination of functionality, reliability, usability and efficiency can be measured externally by quality in use.

6.4.1 Time behaviour

The capability of the software product to provide appropriate response and processing times and throughput rates when performing its function, under stated conditions.

6.4.2 Resource utilisation

The capability of the software product to use appropriate amounts and types of resources when the software performs its function under stated conditions.

NOTE Human resources are included as part of productivity (7.1.2).

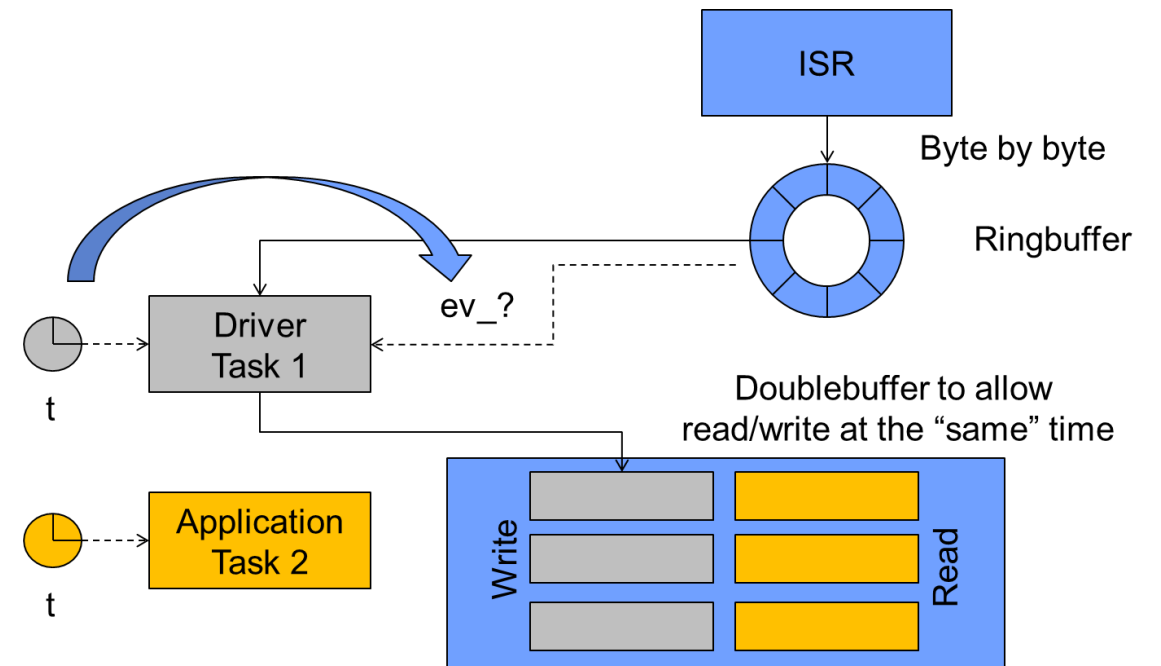
6.4.3 Efficiency compliance

The capability of the software product to adhere to standards or conventions relating to efficiency.

Lidar Example – Efficiency Pitfalls

- Busy waiting approach
- No “parallel” execution of tasks
 - Physical limitation UART 115kBits/s → 10 byte per ms (approx.)
 - 1 scan / 200ms, 400 angels per scan, 5 byte per angle, → 2000 bytes per scan in 200ms = 10 byte per ms

Close to the limit of the CPU???



Maintainability

6.5 Maintainability

The capability of the software product to be modified. Modifications may include corrections, improvements or adaptation of the software to changes in environment, and in requirements and functional specifications.

6.5.1 Analysability

The capability of the software product to be diagnosed for deficiencies or causes of failures in the software, or for the parts to be modified to be identified.

6.5.2 Changeability

The capability of the software product to enable a specified modification to be implemented.

NOTE 1 Implementation includes coding, designing and documenting changes.

NOTE 2 If the software is to be modified by the end user, changeability may affect operability.

6.5.3 Stability

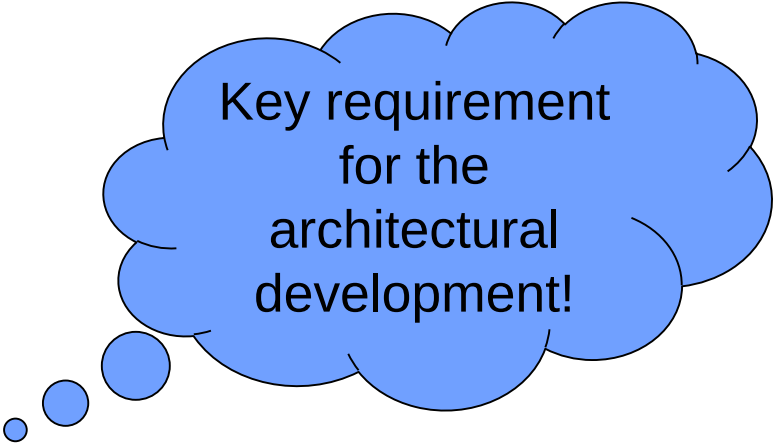
The capability of the software product to avoid unexpected effects from modifications of the software.

6.5.4 Testability

The capability of the software product to enable modified software to be validated.

6.5.5 Maintainability compliance

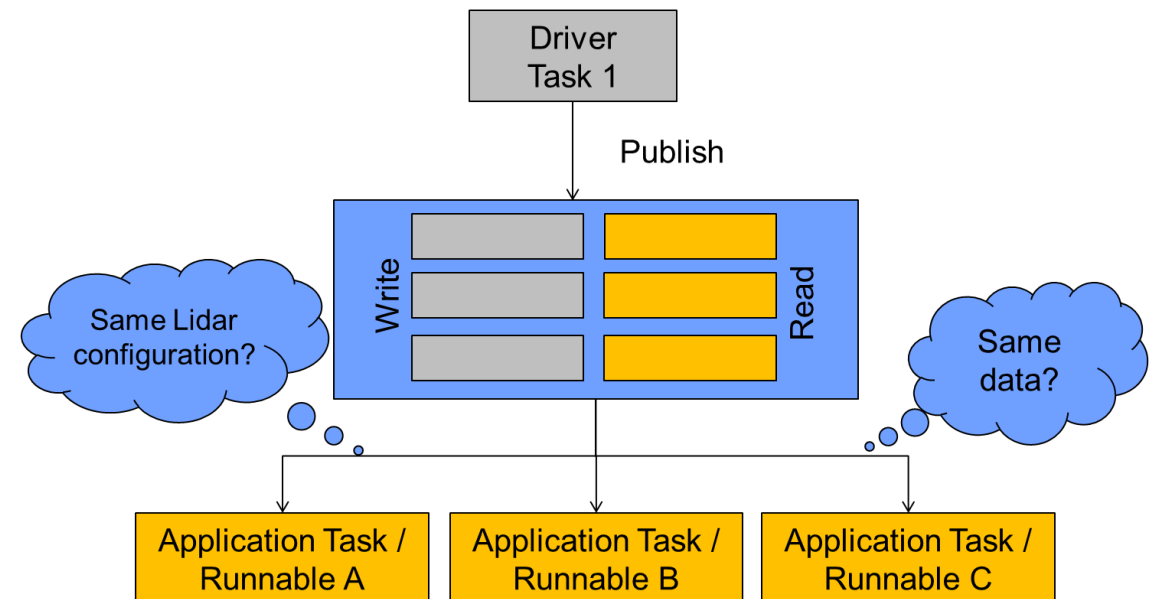
The capability of the software product to adhere to standards or conventions relating to maintainability.

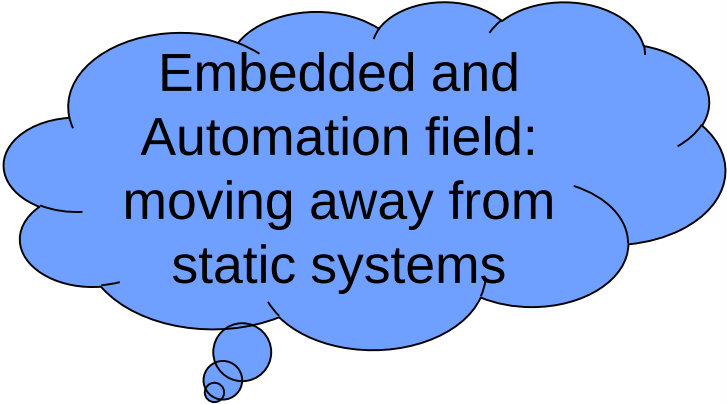


Key requirement
for the
architectural
development!

Lidar Example – Maintainability Pitfalls

- Missing capability to add new services to the system.
- Improper interfacing to CanOpen stack
 - Sniffing the speed packages instead of using CanOpen Client/Server architecture





Embedded and
Automation field:
moving away from
static systems

6.6 Portability

The capability of the software product to be transferred from one environment to another.

NOTE The environment may include organisational, hardware or software environment.

6.6.1 Adaptability

The capability of the software product to be adapted for different specified environments without applying actions or means other than those provided for this purpose for the software considered.

NOTE 1 Adaptability includes the scalability of internal capacity (e.g. screen fields, tables, transaction volumes, report formats, etc.).

NOTE 2 If the software is to be adapted by the end user, adaptability corresponds to suitability for individualisation as defined in ISO 9241-10, and may affect operability.

6.6.2 Installability

The capability of the software product to be installed in a specified environment.

NOTE If the software is to be installed by an end user, installability can affect the resulting suitability and operability.

6.6.3 Co-existence

The capability of the software product to co-exist with other independent software in a common environment sharing common resources.

6.6.4 Replaceability

The capability of the software product to be used in place of another specified software product for the same purpose in the same environment.

NOTE 1 For example, the replaceability of a new version of a software product is important to the user when upgrading.

NOTE 2 Replaceability is used in place of **compatibility** in order to avoid possible ambiguity with interoperability (see 6.1.3).

NOTE 3 Replaceability may include attributes of both installability and adaptability. The concept has been introduced as a subcharacteristic of its own because of its importance.

6.6.5 Portability compliance

The capability of the software product to adhere to standards or conventions relating to portability.



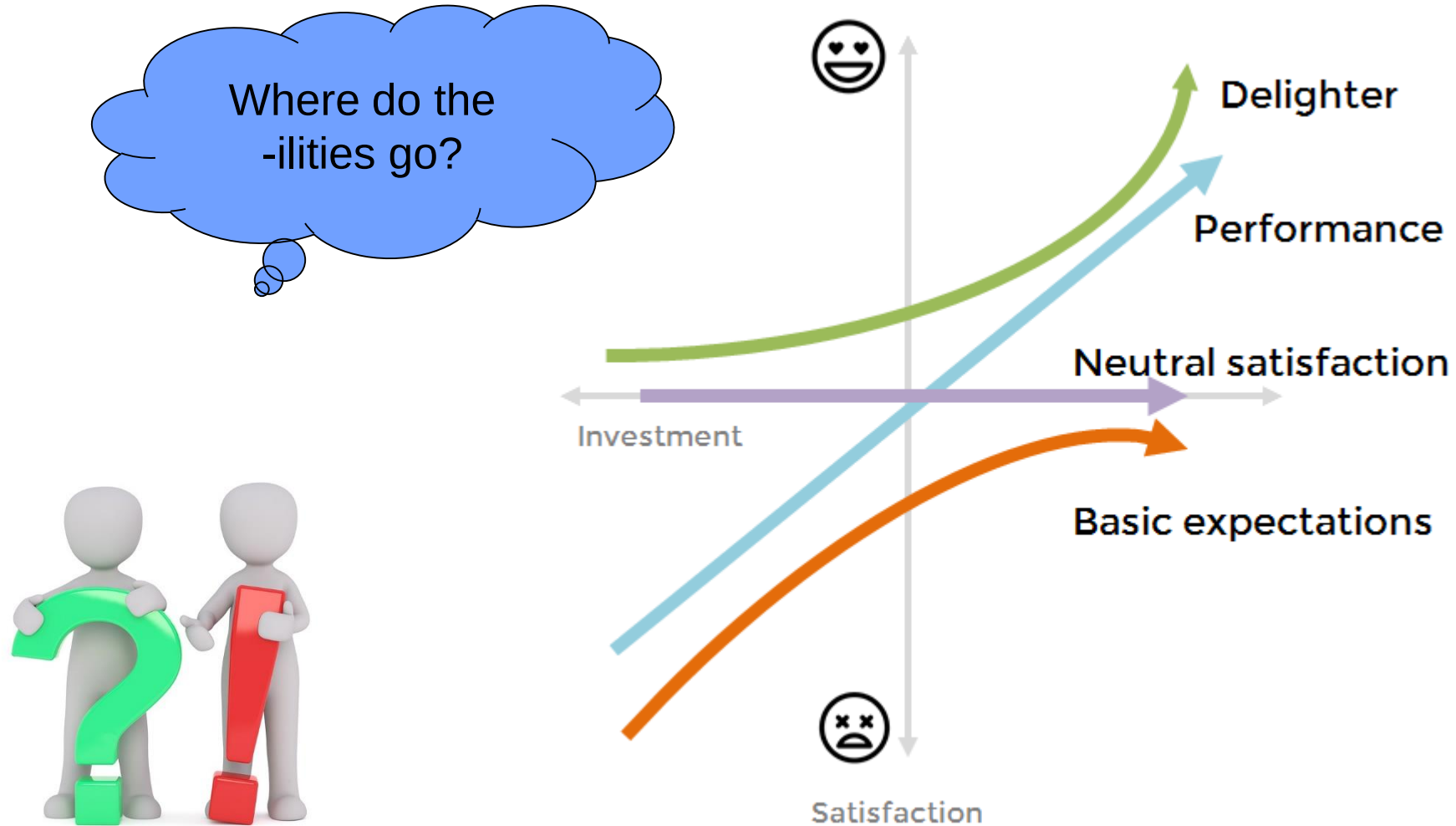
Standards
help!

Lidar Example – Portability Pitfalls

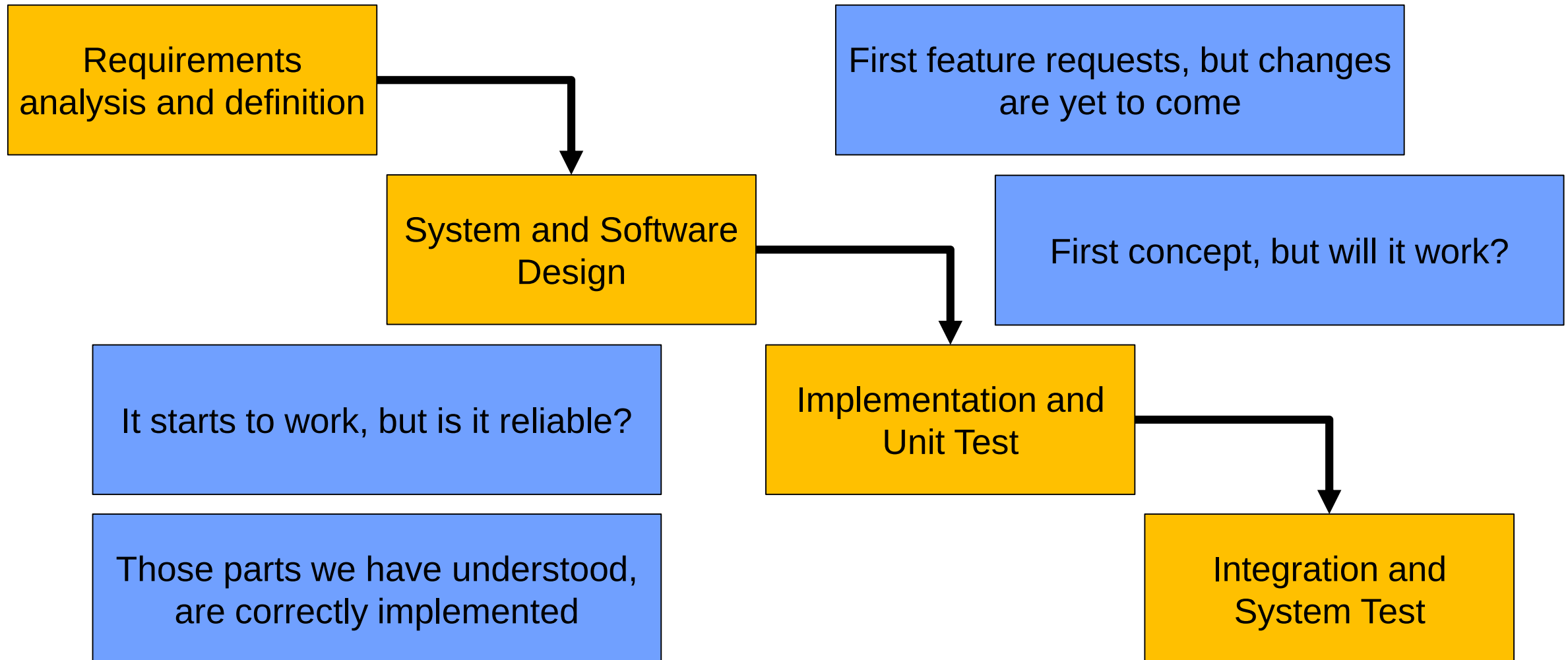
- In a recent project, the team wanted to use the Lidar board as an additional gateway for camera data.
- How about the system load – can we add another service?
- What kind of side effects do we expect?
- ➔ Hacky solution – let's take another board.
In industry way too expensive!!!!
- How about porting the firmware to a more powerful controller.
 - Hardware abstraction
 - OS abstraction



Making the User happy - KANO Model



How much do we truly know?



Key problem of Engineering



Once we ARE done,
we know HOW and
WHAT we should
have done!

How can we solve this problem?

Obviously, this
will not serve as
an excuse!

Try to get the critical
parts done as early
as possible!

Be willing to refactor
it!



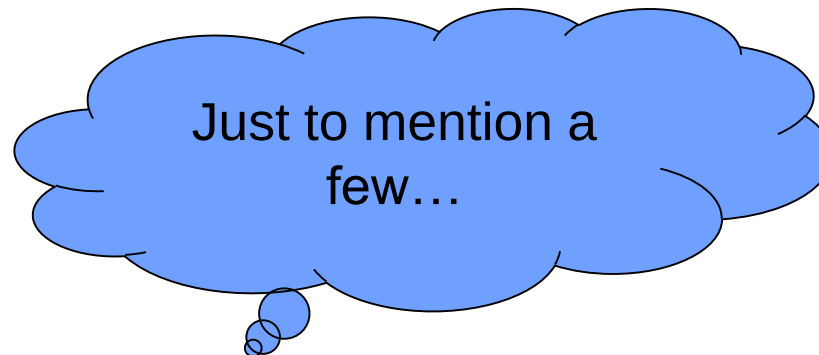
Why Refactoring?

Code refactoring is the process of restructuring existing computer code—changing the factoring—without changing its external behavior. Refactoring is intended to improve nonfunctional attributes of the software. Advantages include improved code readability and reduced complexity; these can improve source-code maintainability and create a more expressive internal architecture or object model to improve extensibility.

Wikipedia

Techniques

- Restructuring
- Encapsulation
- Renaming, commenting
- Data flow improvement
- Introduction of code and design patterns
- Rework dangerous parts
- Improving error handling



Chances and risks of Refactoring

Start early! Take
your time! Assess
risks / benefit!

Use version
management /
nightly build

Worse structure

New bugs

Improved structure /
better maintenance
/ performance

Documentation /
code not in sync

Use tool support
(Eclipse, Beyond
Compare)

Cost explosion!

Use models for
early design
verification

Refactoring risks

No risk

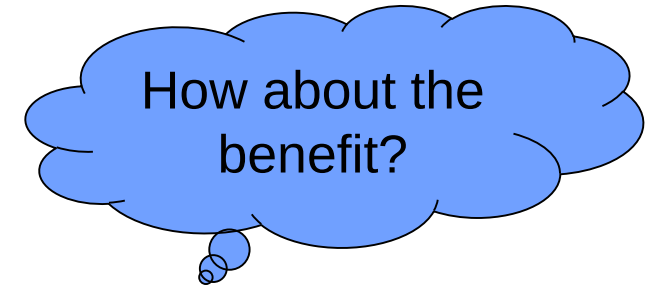
- Adding comments
- Changing variable and function / class names
- Adding encapsulation (namespaces)

Medium risk

- Improving functions (not affecting the public API)
- Switch from C to C++ compiler (for stricter type-checking)
- Changing task / interrupt priorities

High risk

- Adding hardware features like MPU
- Changing data flow based on globals
- Changing public function API
- Changing architectural concepts
 - Error handling
 - Notification concepts



Wrap-Up

- Difference between Features and Structure
- IEC9126 Quality Model
 - Functionality
 - Reliability
 - Usability
 - Maintainability
 - Portability
 - Efficiency
- Incremental, Iterative development
- KANO-Model
- Motivation and Need for Refactoring

We don't have
enough time to
write bad code!

How do I write Clean Code?

I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code

